

Undecidability

Reading: Chapter 8 & 9

Decidability vs. Undecidability

- There are two types of TMs (based on halting):

(*Recursive*)

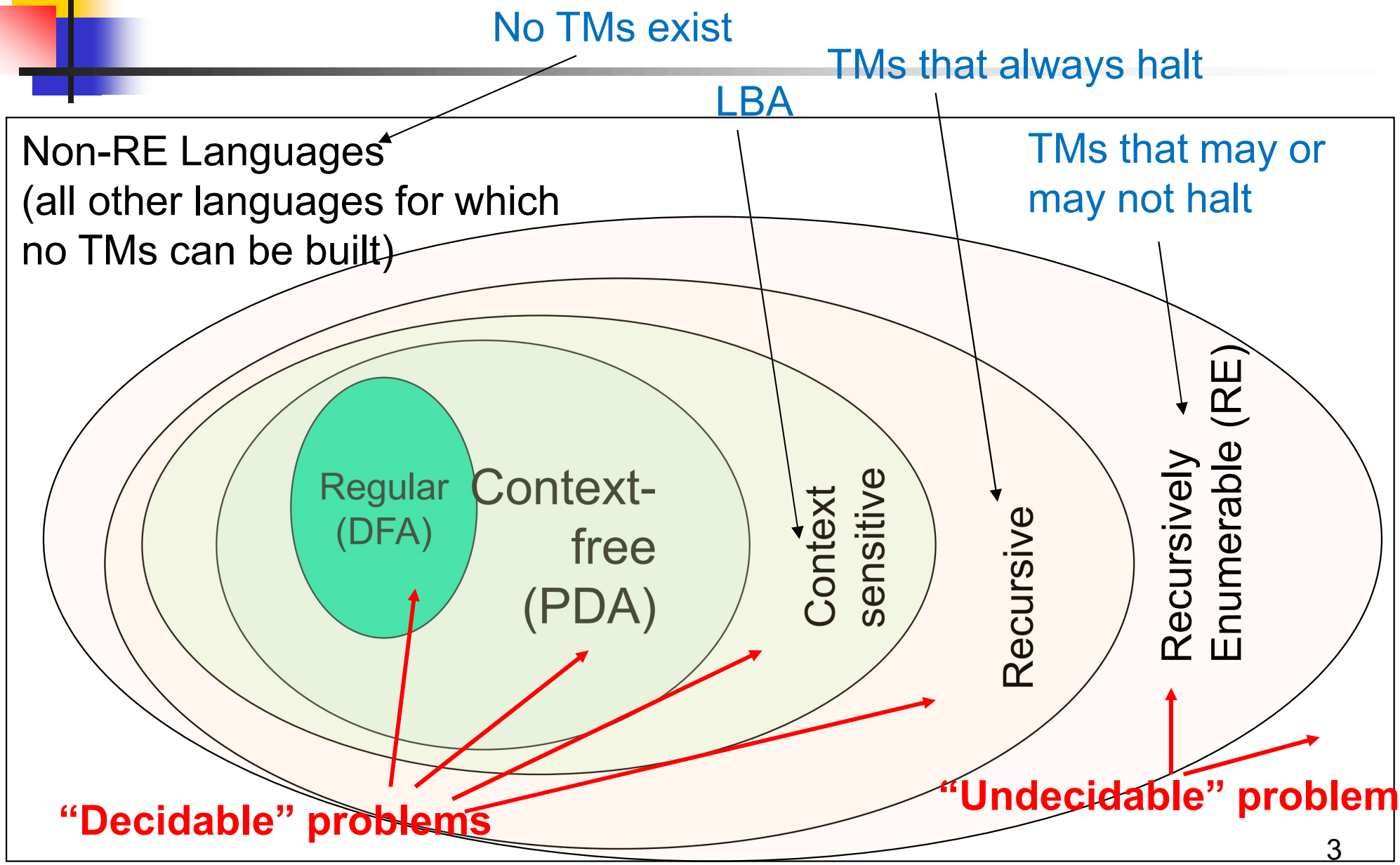
TMs that *always* halt, no matter accepting or non-accepting ≡ **DECIDABLE** PROBLEMS

(*Recursively enumerable*)

TMs that *are guaranteed to halt* only on acceptance. If non-accepting, it may or may not halt (i.e., could loop forever).

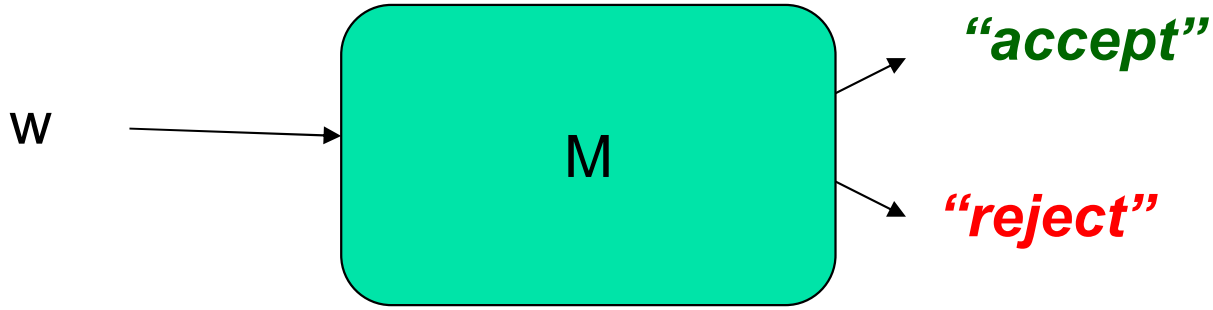
- **Undecidability:**
  - Undecidable problems are those that are not recursive

Recursive, RE, Undecidable languages

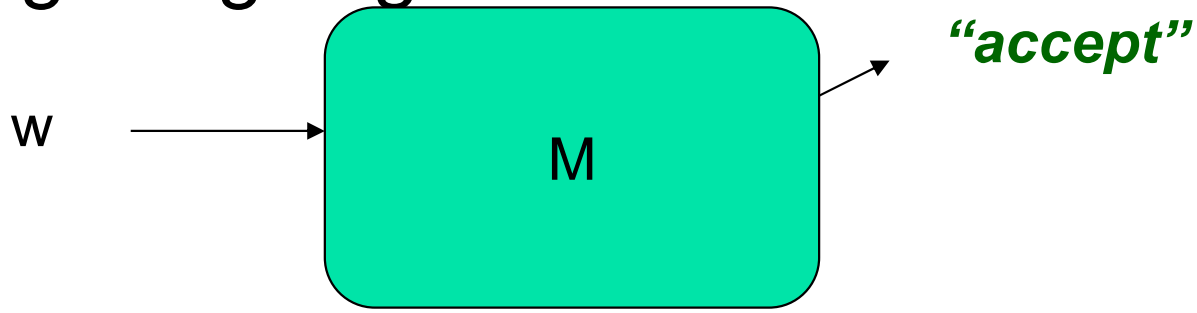


Recursive Languages & Recursively Enumerable (RE) languages

- Any TM for a Recursive language is going to look like this:



- Any TM for a Recursively Enumerable (RE) language is going to look like this:



“Languages” vs. “Problems”

A “language” is a set of strings

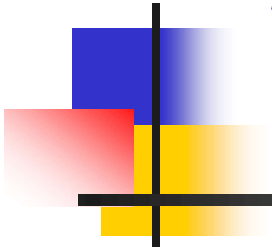
Any “problem” can be expressed as a set of all strings that are of the form:

- “<input, output>”

e.g., Problem (a+b) ≡ Language of strings of the form { “a#b, a+b” }

==> Every problem also corresponds to a language!!

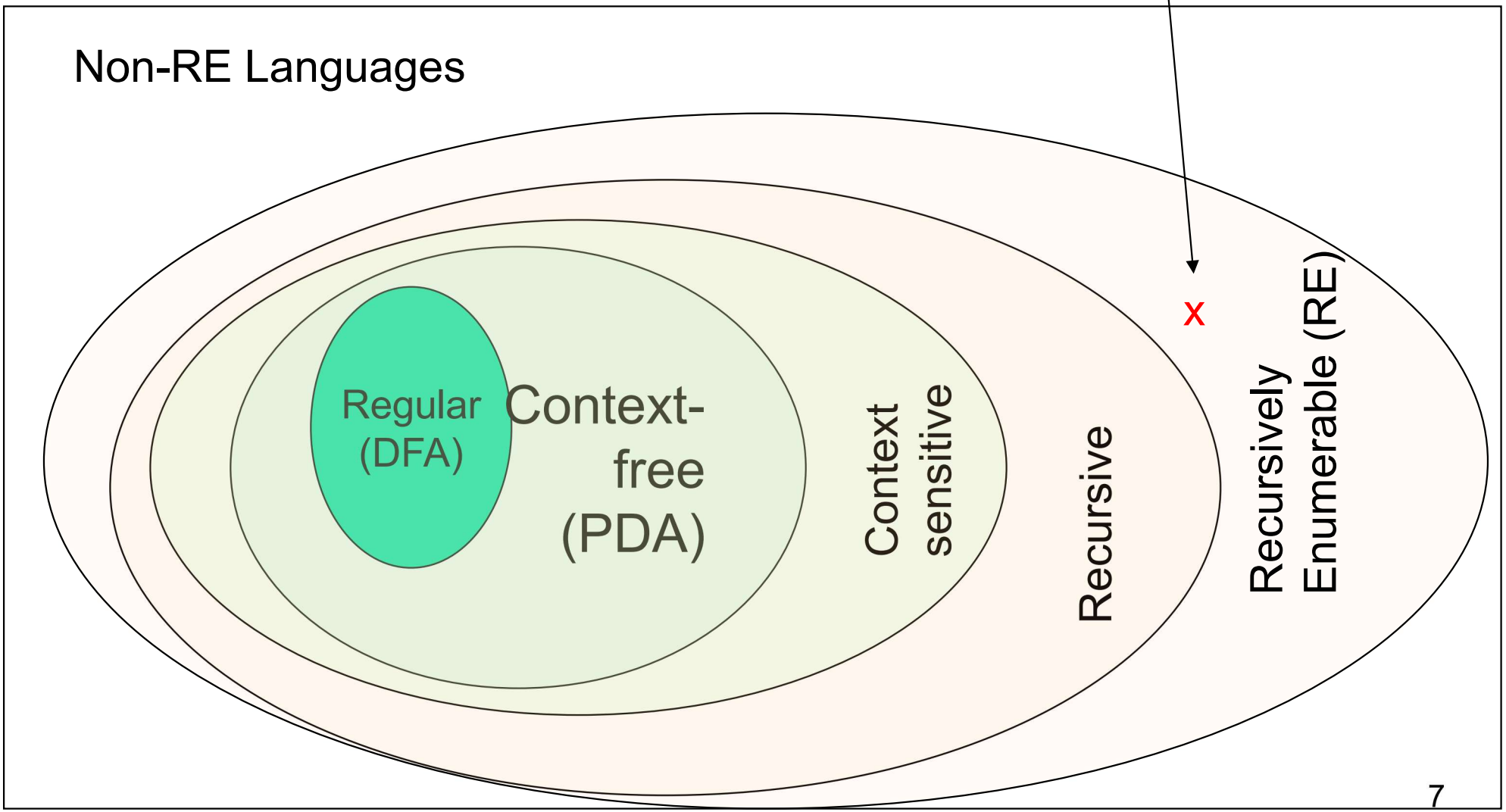
Think of the language for a “problem” == a *verifier* for the problem



The Halting Problem

An example of a recursive enumerable problem that is also undecidable

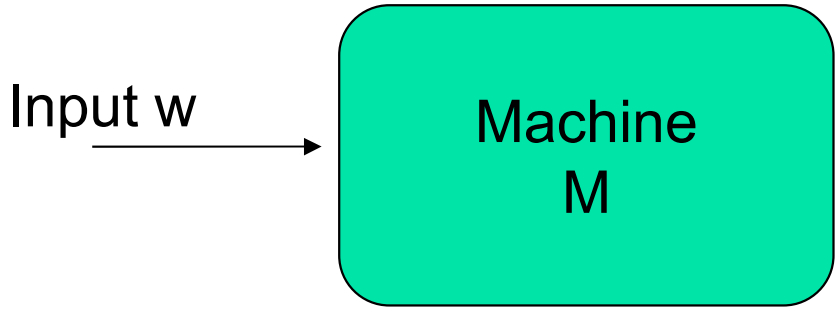
The Halting Problem



What is the Halting Problem?

Definition of the “halting problem”:

- *Does a givenTuring Machine M halt on a given input w?*



A Turing Machine simulator

The Universal Turing Machine

- Given: TM **M** & its input **w**
- Aim: Build another TM called "H", that will output:
  - "*accept*" if M accepts w, and
  - "reject" otherwise

- An algorithm for H:
  - Simulate M on w

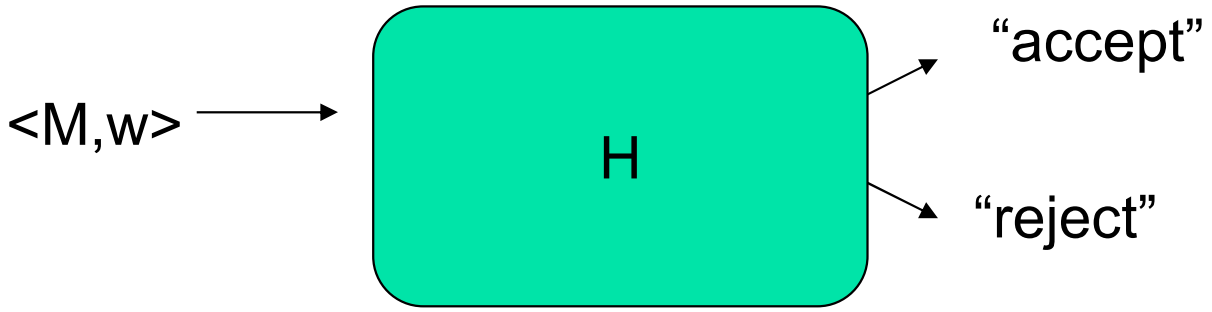
Implies: H is in RE

  - H(<M,w>) =
    - *accept*, if *M* accepts w
    - *reject*, if M does does not accept w

Question: If M does *not* halt on w, what will happen to H?

A Claim

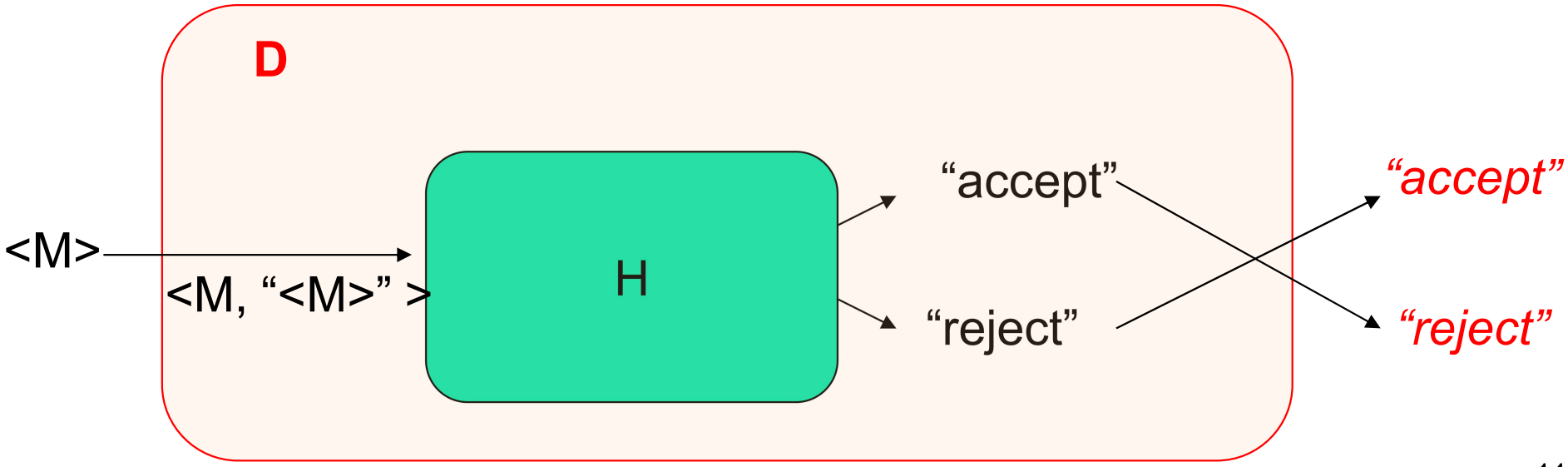
- Claim: No H that is always guaranteed to halt, can exist!
- Proof: (Alan Turing, 1936)
  - By contradiction, let us assume H exists



Therefore, if H exists ➔ D also should exist.
But can such a D exist? (if not, then H also cannot exist)

HP Proof (step 1)

- Let us construct a new TM **D** using H as a subroutine:
  - On input <M>:
    1. Run H on input <M, <M> >; //(i.e., run M on M itself)
    2. Output the *opposite* of what H outputs;



HP Proof (step 2)

- The notion of inputing "<M>" to M itself
  - A program can be input to itself (e.g., a compiler is a program that takes any program as input)

D (<M>) = { *accept*, if M does *not* accept <M>; *reject*, if M accepts <M>

Now, what happens if D is input to itself?

D (<D>) = { *accept*, if D does not accept <D>; *reject*, if D accepts <D>

A contradiction!!! ==> Neither D nor H can exist.

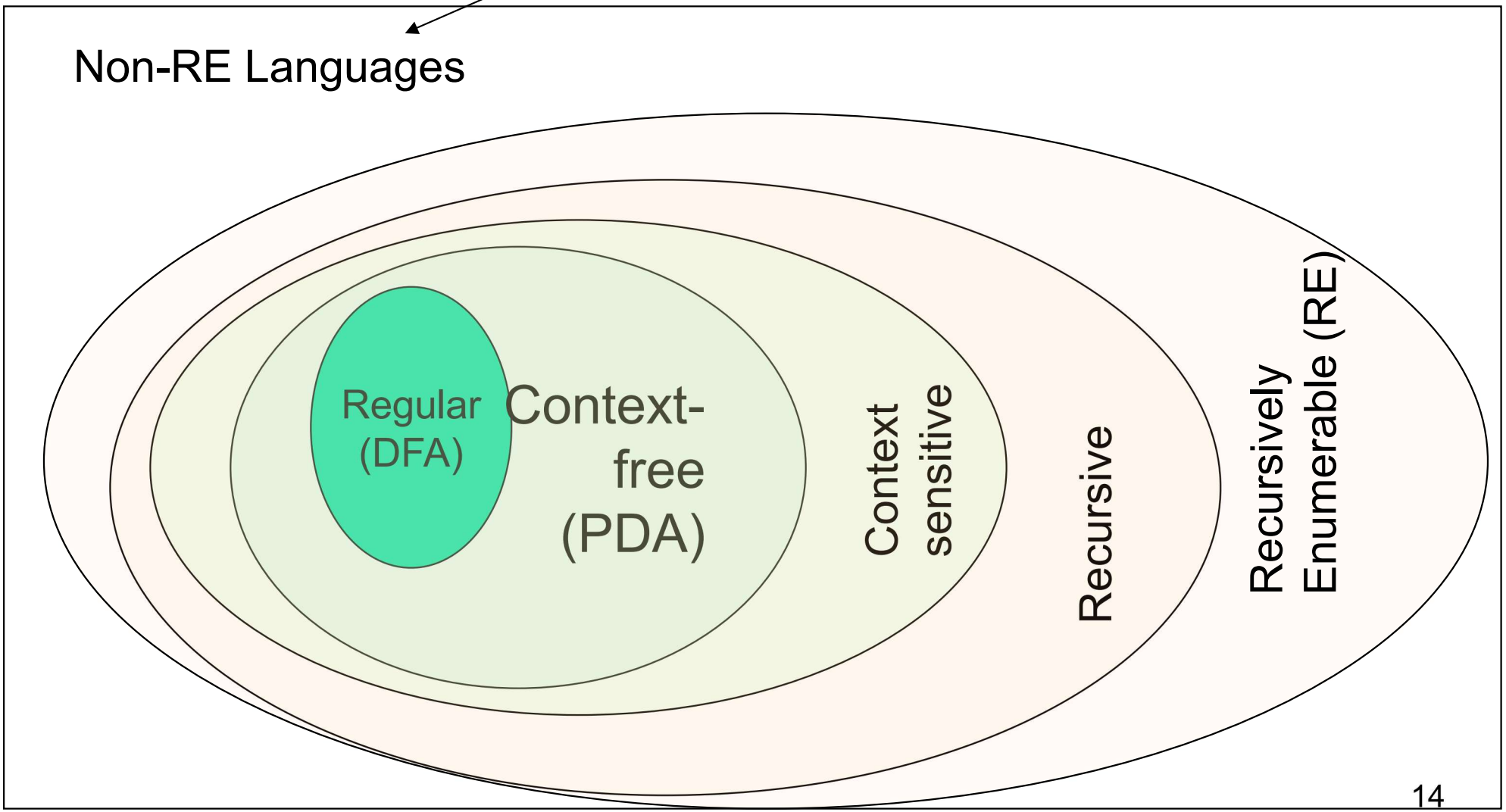
Why should there be languages that do not have TMs?

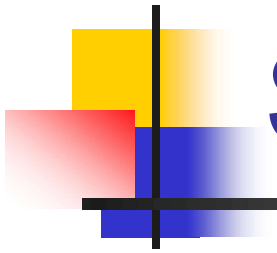


We thought TMs can solve everything!!

Non-RE languages

How come there are languages here?
(e.g., diagonalization language)





Summary

- Problems vs. languages
- Decidability
  - Recursive
- Undecidability
  - Recursively Enumerable
  - Not RE
- Reducability